

CSE4067

Introduction to Blockchain Programming

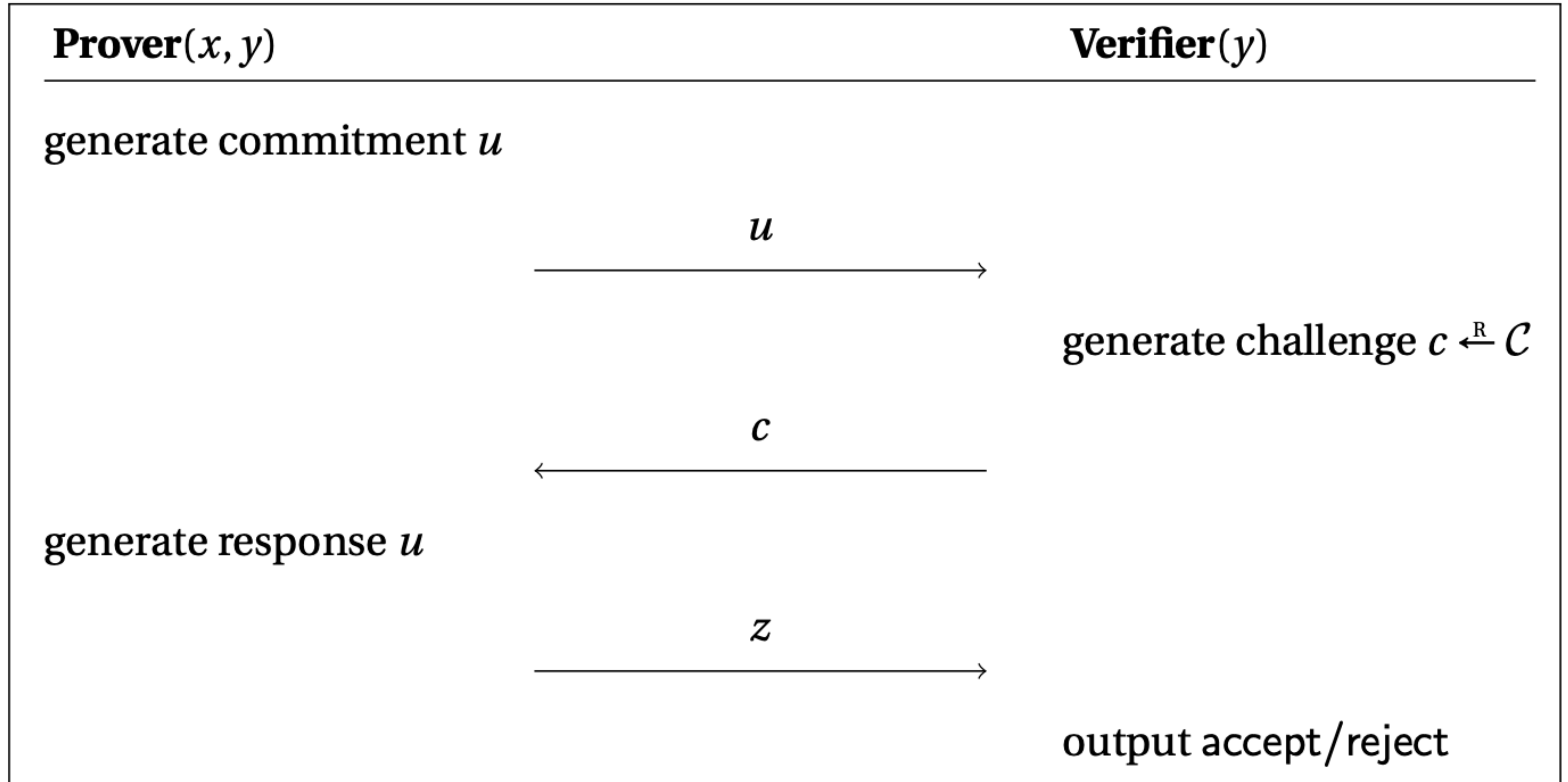
Goshgar Can Ismayilov

goshgar.ismayilov@boun.edu.tr

<https://www.linkedin.com/in/gismayilov/>

24.12.2024

Sigma Protocol



Fiat-Shamir Protocol

Integer Factorization

Protocol 5.1 (Fiat-Shamir Identification Protocol [4]).

Initialization

1. A trusted center T selects and publishes an RSA-like modulus $n = pq$ but keeps the primes p and q secret.
2. The prover selects a secret s coprime to n , $1 \leq s \leq n - 1$, computes $v = s^2 \bmod n$, and registers v with T as her public key.

Fiat-Shamir Protocol

Identification Protocol

The following steps are executed t times, each time using independent random coin tosses.

- (P1) The prover chooses a random r , $1 \leq r \leq n - 1$ and sends $x = r^2 \bmod n$ to the verifier.
- (V1) The verifier randomly selects a bit $e \in \{0, 1\}$ and sends e to the prover.
- (P2) The prover computes and sends to the verifier y , where $y = r$ (if $e = 0$) or $y = rs \bmod n$ (if $e = 1$).
- (V2) The verifier *rejects* if $y = 0$ or if $y^2 \not\equiv x \cdot v^e \pmod{n}$. (Depending on e , $y^2 = x$ or $y^2 = xv \bmod n$, since $v = s^2 \bmod n$. Note that checking for $y = 0$ precludes the case $r = 0$.)

Fiat-Shamir Protocol

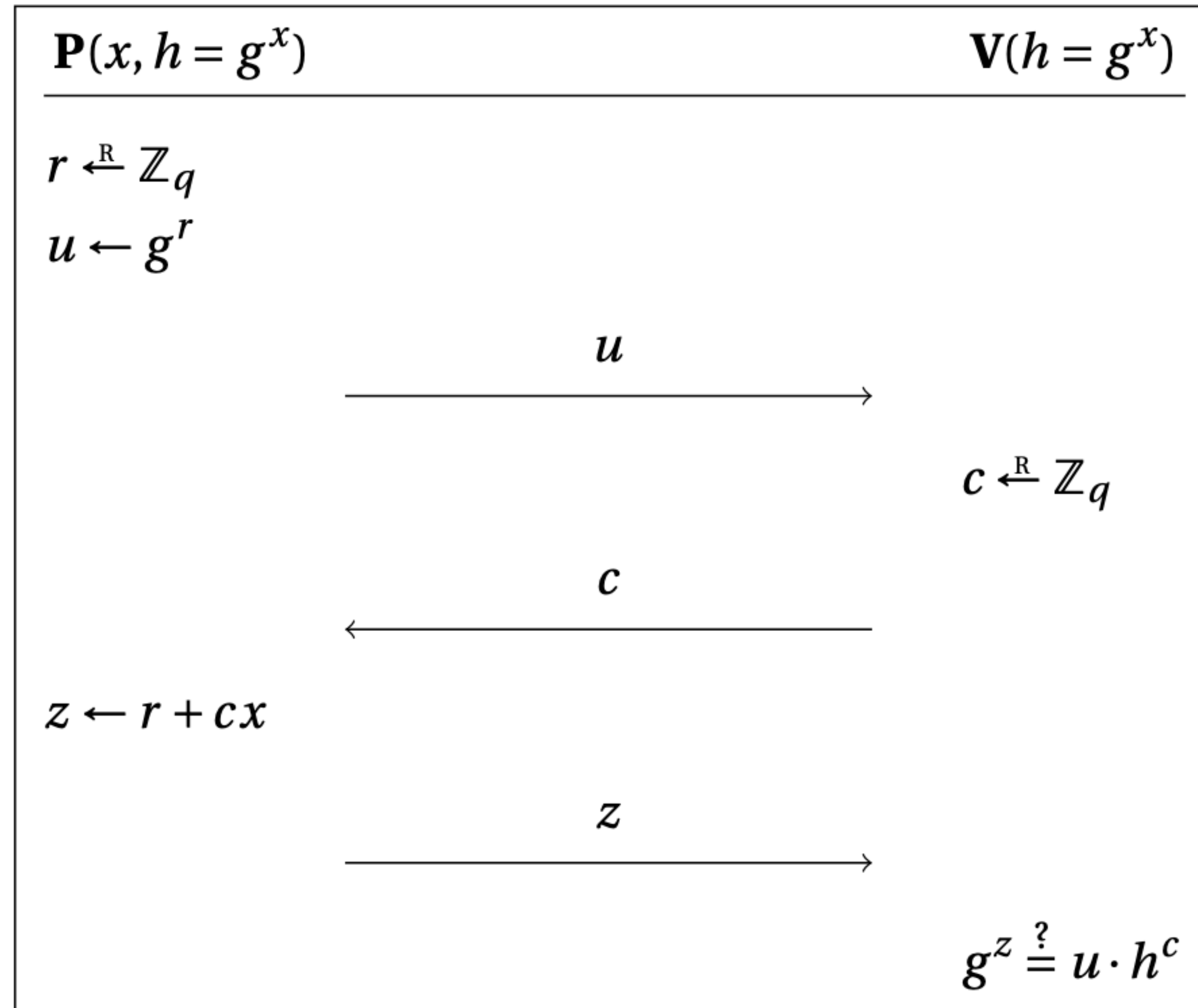
Remark 5.3. As in the cave story, one might wonder why Victor doesn't always request $y = rs$, thus forcing Peggy to make use of s . If this is done, Peggy has a cheating strategy. She can select r randomly and then set $x = \frac{r^2}{v}$. When Victor requests $y = rs$, Peggy will send $y = r$. Now, when Victor attempts to verify Peggys response, he will compute:

$$x \cdot v^e \equiv \frac{r^2}{v} \cdot v \equiv r^2 \equiv y^2 \pmod{n}$$

thus fooling Victor into believing she has possession of s . This strategy becomes ineffective if Victor asks either of the two questions at random.

Schnorr's Protocol

Discrete Log

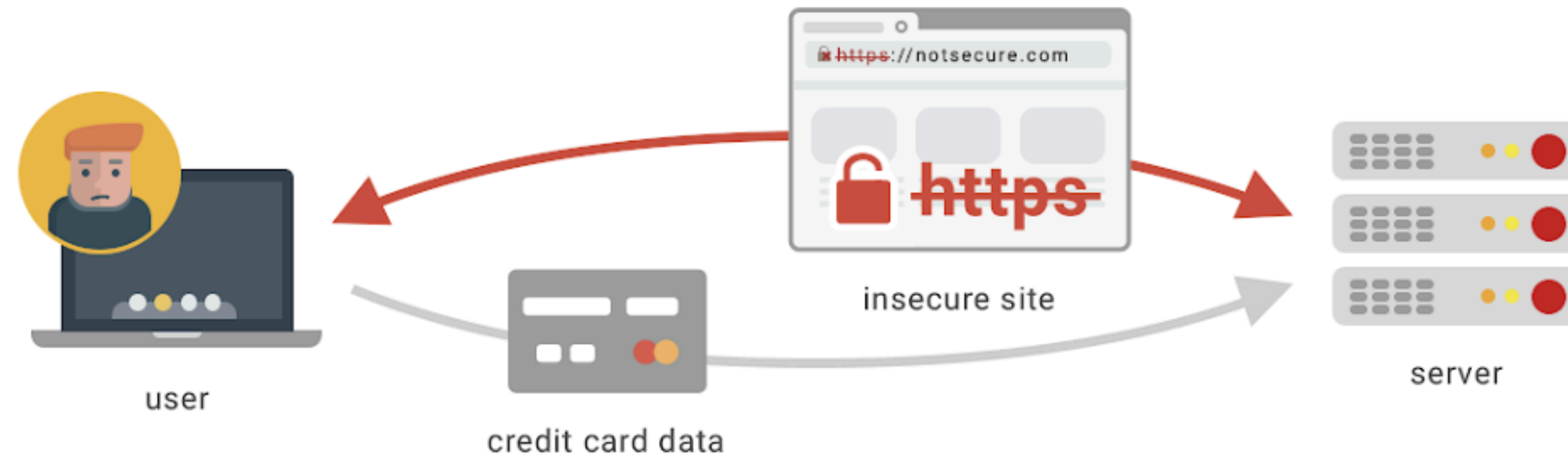


$$g^z \stackrel{?}{=} u \cdot h^c$$

Zcash

HTTP vs HTTPS

Credit Card information over HTTP



What a Hacker sees

All data in cleartext

Credit Card information over HTTPS

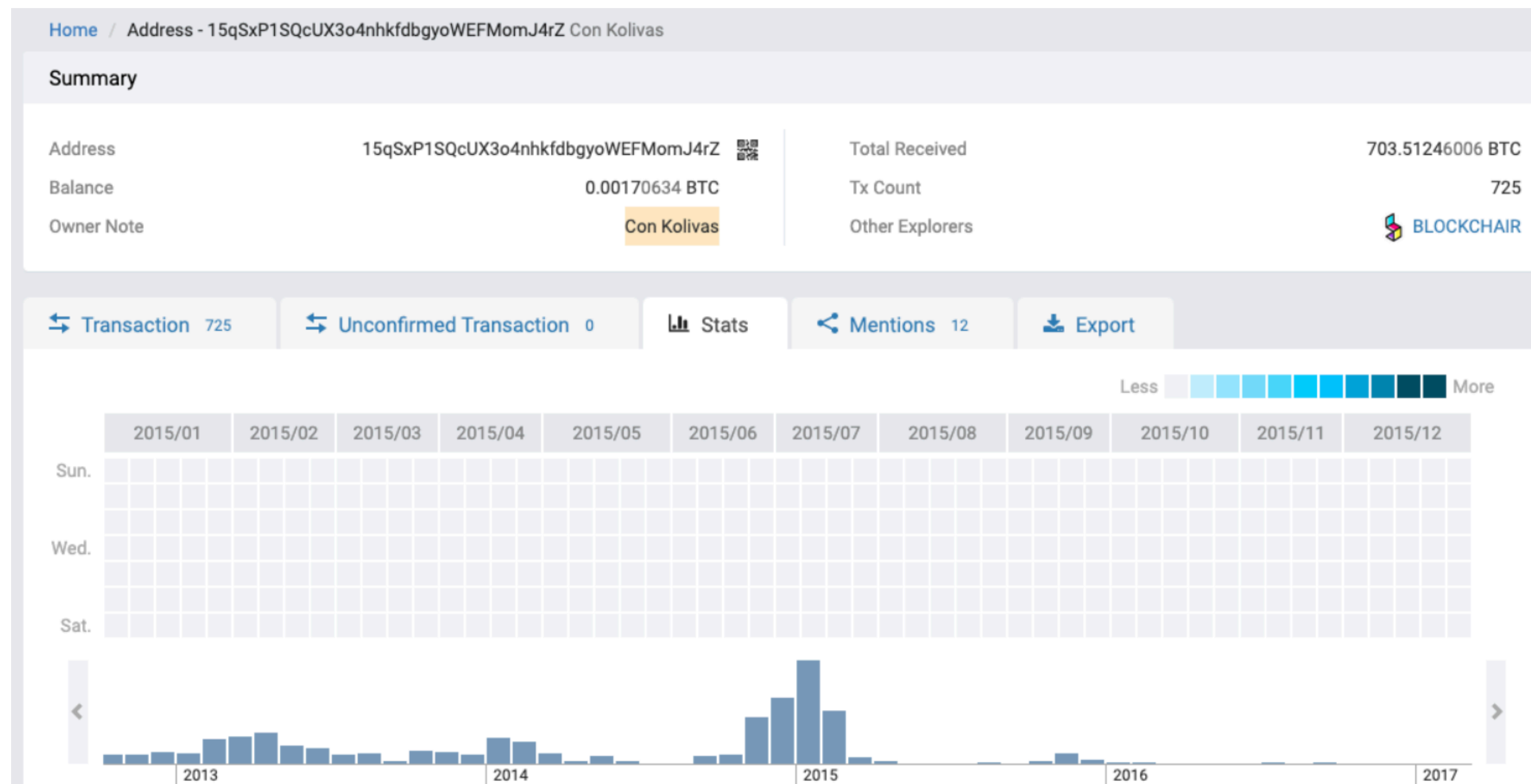


What a Hacker sees

All data encrypted

Zcash

To make this concrete, consider the relatively harmless case of crypto developer Con Kolivas, who accepted public donations for his open source work through the Bitcoin address [15qSxP1SQcUX3o4nhkfdbgyoWEFMomJ4rZ](https://blockchain.info/address/15qSxP1SQcUX3o4nhkfdbgyoWEFMomJ4rZ). His name is now mapped to that address, and recorded in block explorers like BTC.com. As [shown below](#), the total amount Kolivas received in donations (703.51246006 BTC), the exact amount sent in each donation, and the number and timestamps of all donations are all permanently world-readable.



Zcash

That's why [Zcash](#) was invented in 2016. Zcash uses a cryptographic breakthrough called [zero-knowledge proofs](#) layered on top of the Bitcoin protocol, to provide a means for nodes on the network to verify that a transaction is valid – without giving them any information about the transaction, including sender, receiver, or transaction amount. If we think of a blockchain as a public ledger, fully shielded Zcash transactions don't print the from or to addresses of each transaction, or the amounts.

Zcash

Public ledger of transactions

Zcash shielded transactions

vs

Bitcoin transactions

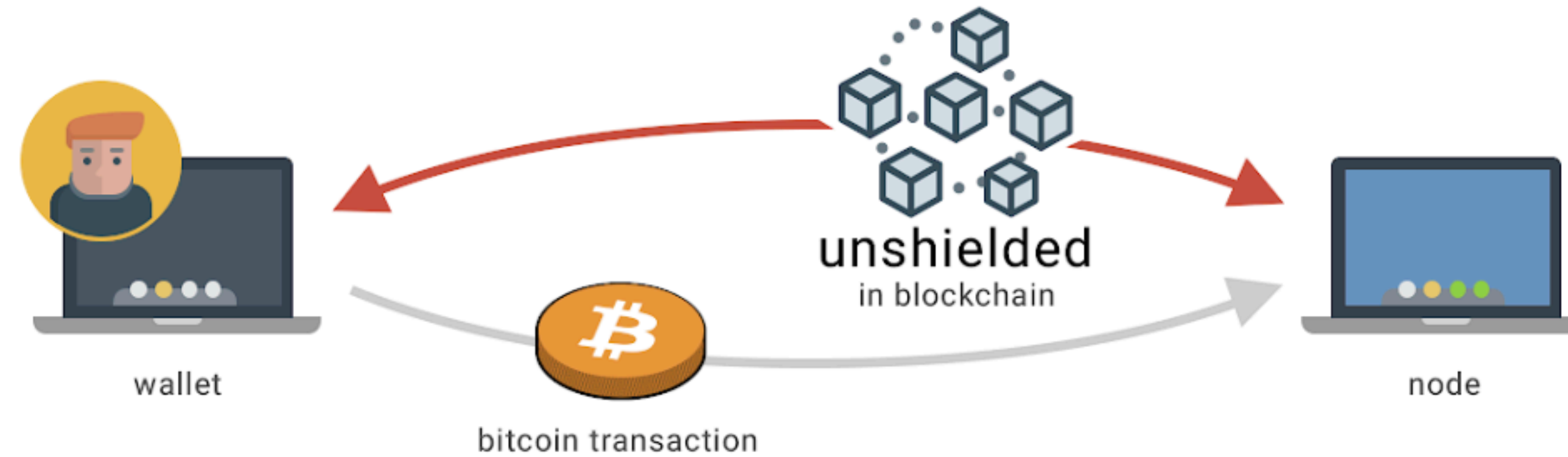
Block	Sender	Recipient	Amount	Proof
36809	Encrypted	Encrypted	Encrypted	a6b5...
38223	Encrypted	Encrypted	Encrypted	d89c...
98001	Encrypted	Encrypted	Encrypted	f11a...

Block	Sender	Recipient	Amount
36809	Address A	Address B	3 BTC
38223	Address B	Address C	2 BTC
98001	Address C	Address D	1 BTC

Zcash

Unshielded vs Shielded Transactions

Unshielded blockchain transactions

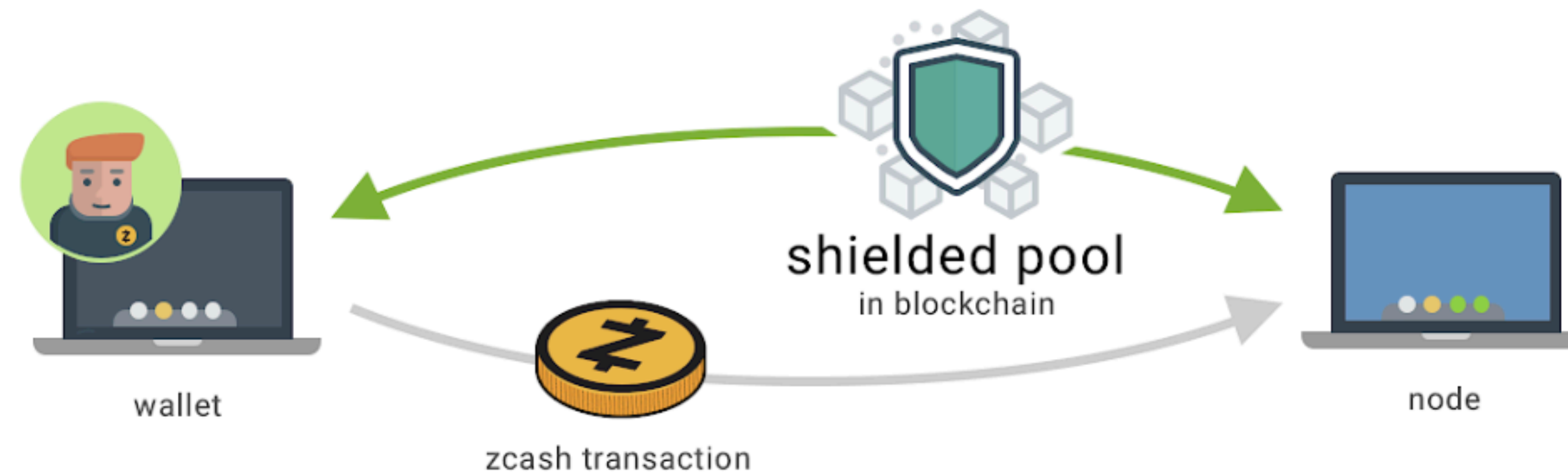


What a Hacker sees

```
BTC_Donate=3CMCRgE  
HVz...  
jcl...  
=&from=1AEwvN9YdR5  
keNzSThGXFpsNWQxgn  
SaxHmWiJXpL&return  
&amount=2500.00&r  
ipient_address=17h  
NzSThGXFpsNWQxgnkY  
xHmWiJXpL&to=377SX  
yt:
```

All data in cleartext

Shielded blockchain transactions



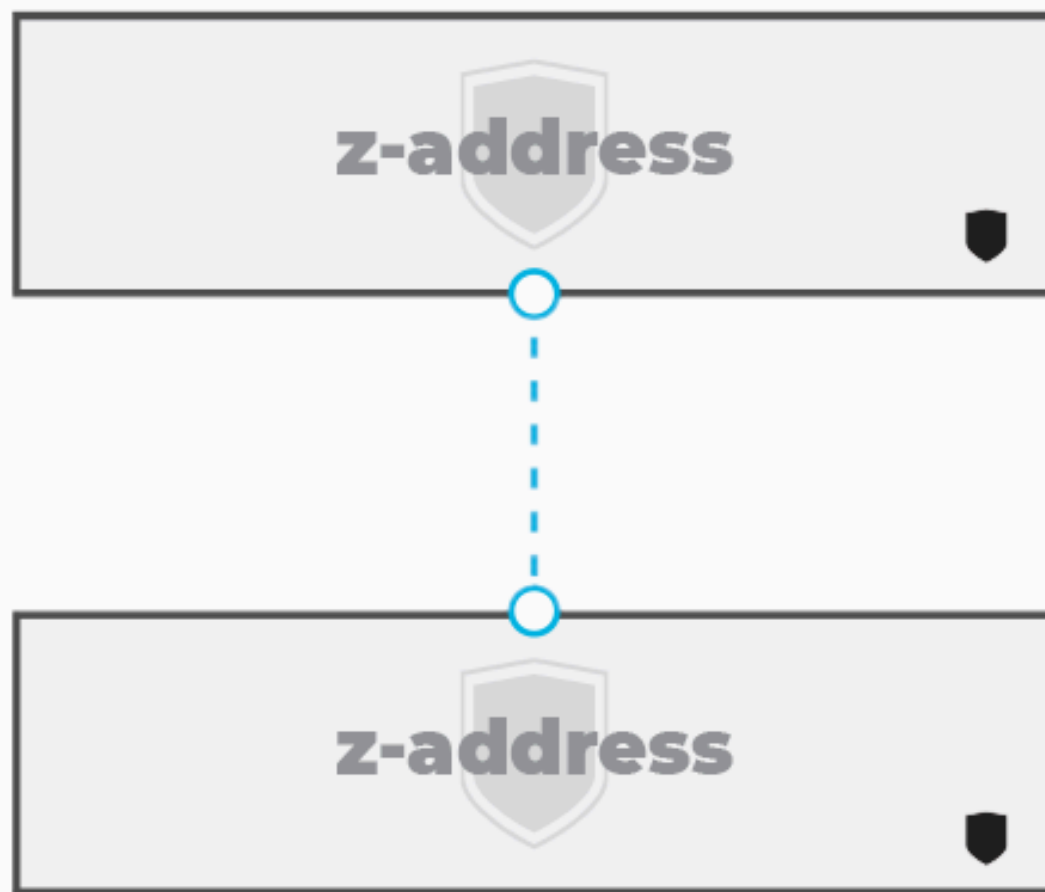
What a Hacker sees



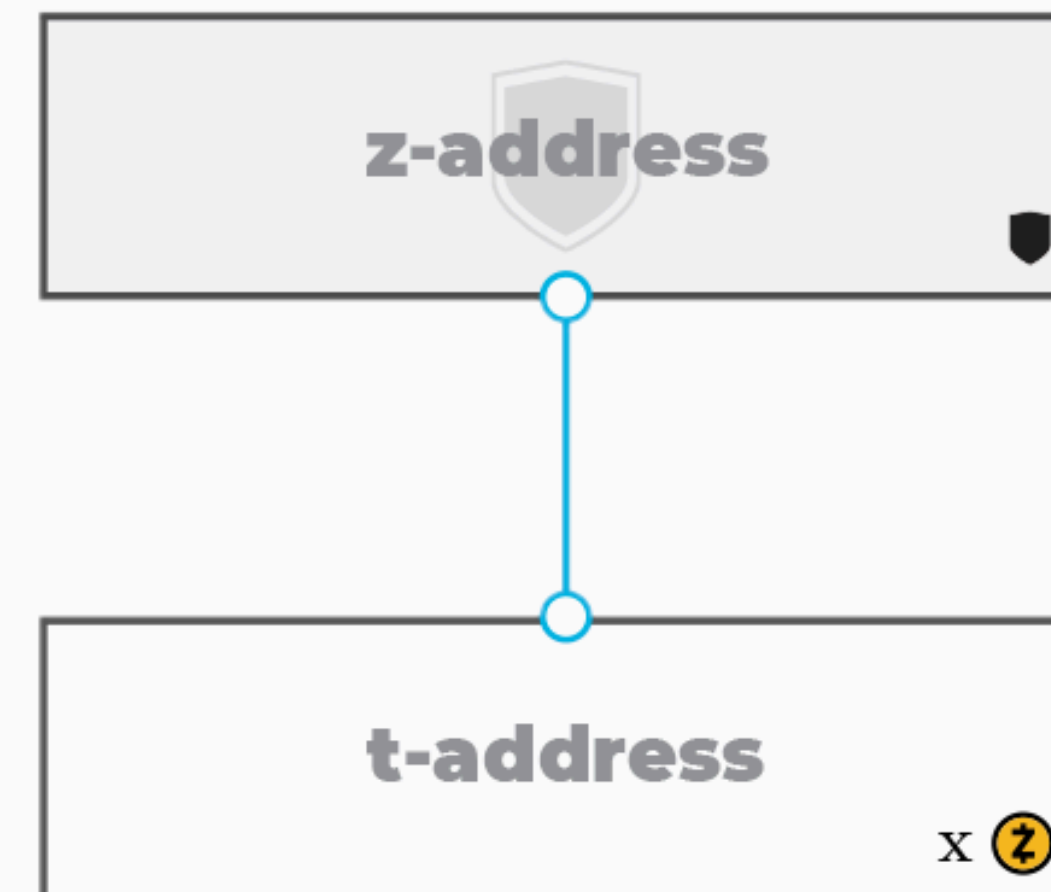
All data encrypted

Zcash

Zcash has two main types of addresses: shielded (z-addresses which start with “z”) and transparent (t-addresses which start with “t”).



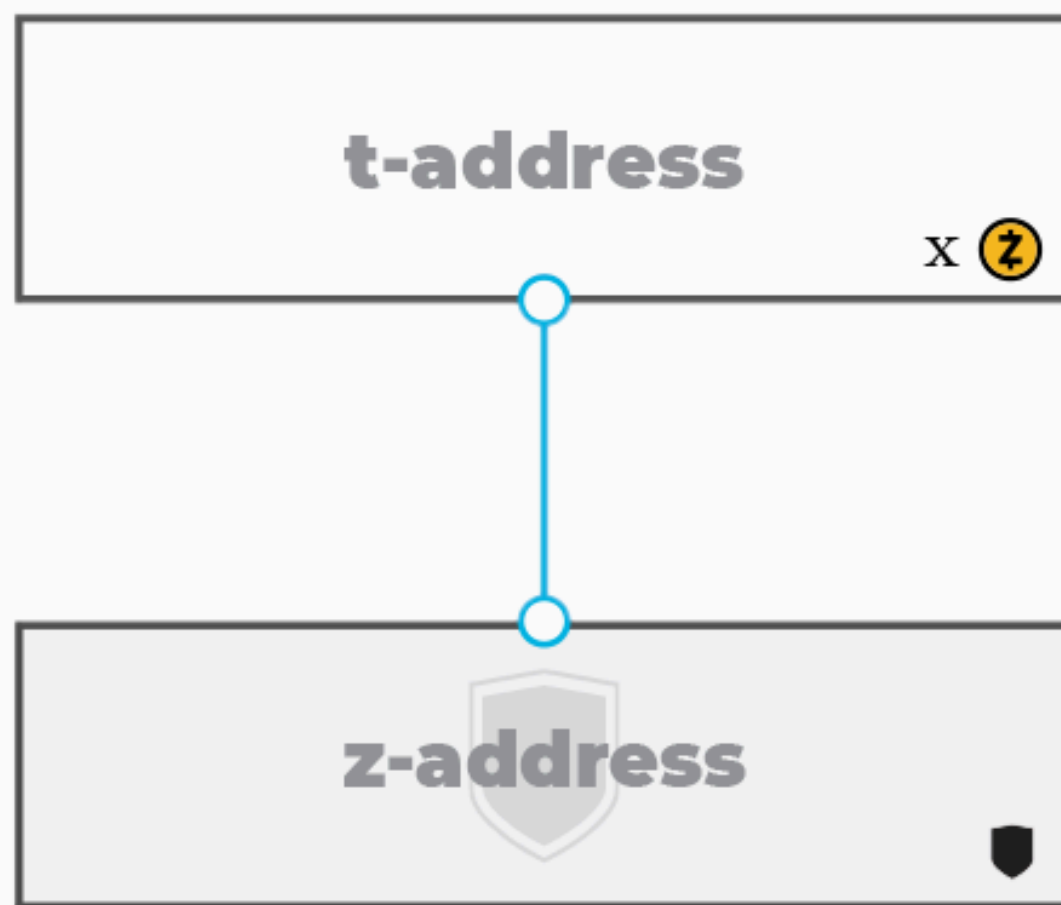
Shielded/private (Value is not revealed on the blockchain) 🔒



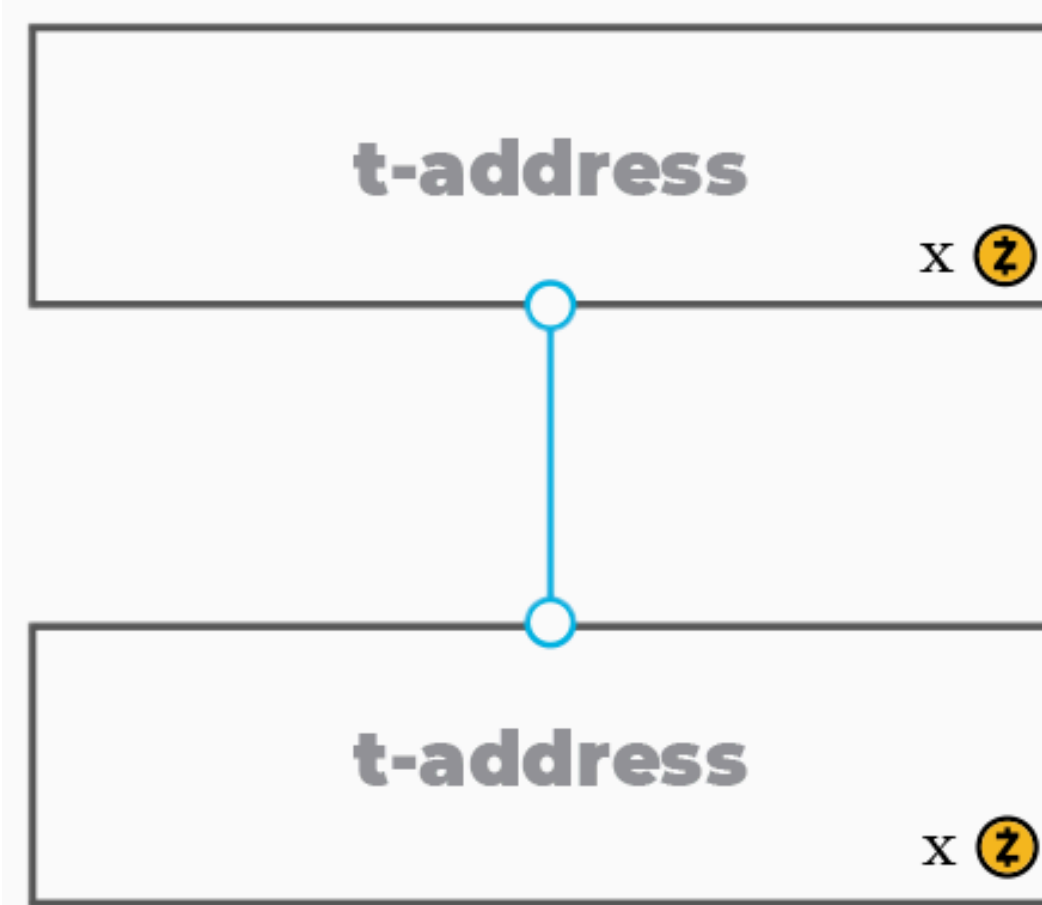
Deshielding (Value is revealed on the receiver end) 🔒

Zcash

Zcash has two main types of addresses: shielded (z-addresses which start with “z”) and transparent (t-addresses which start with “t”).

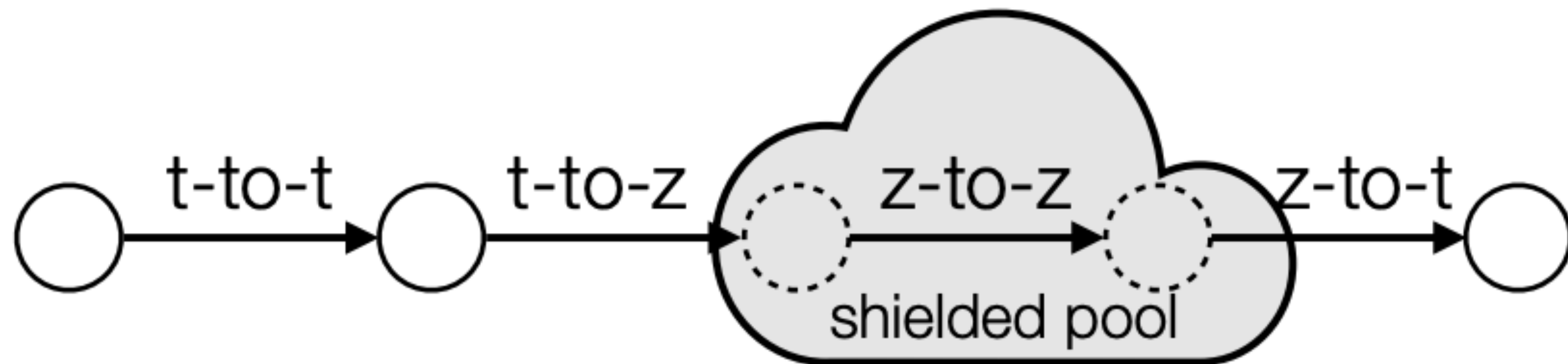
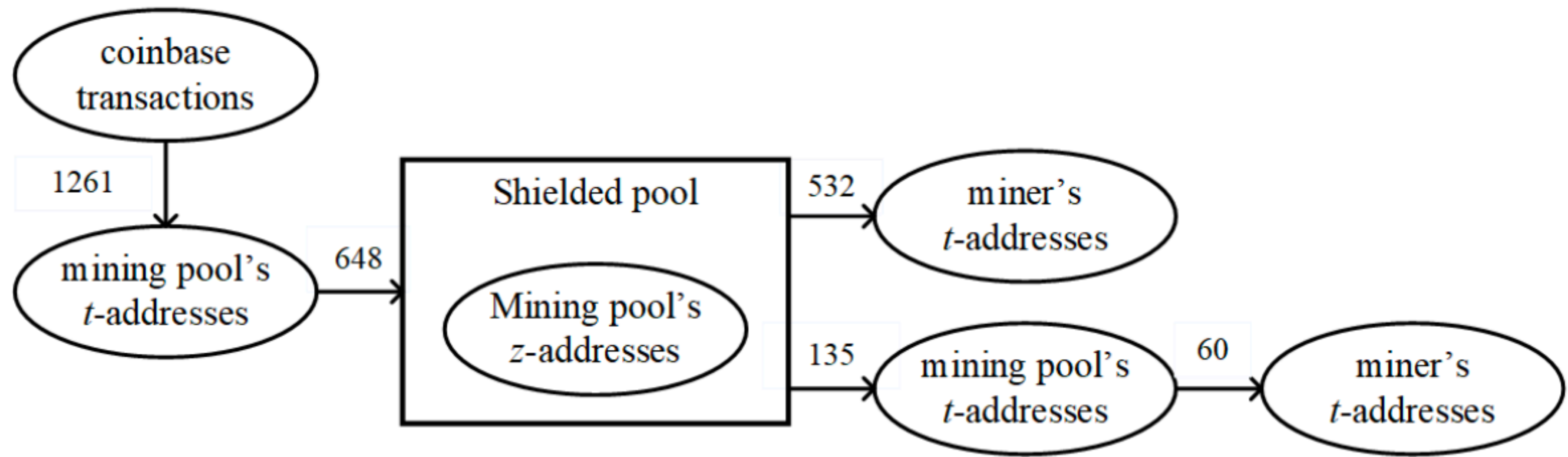


Shielding (Value is revealed on the sender end)🔵

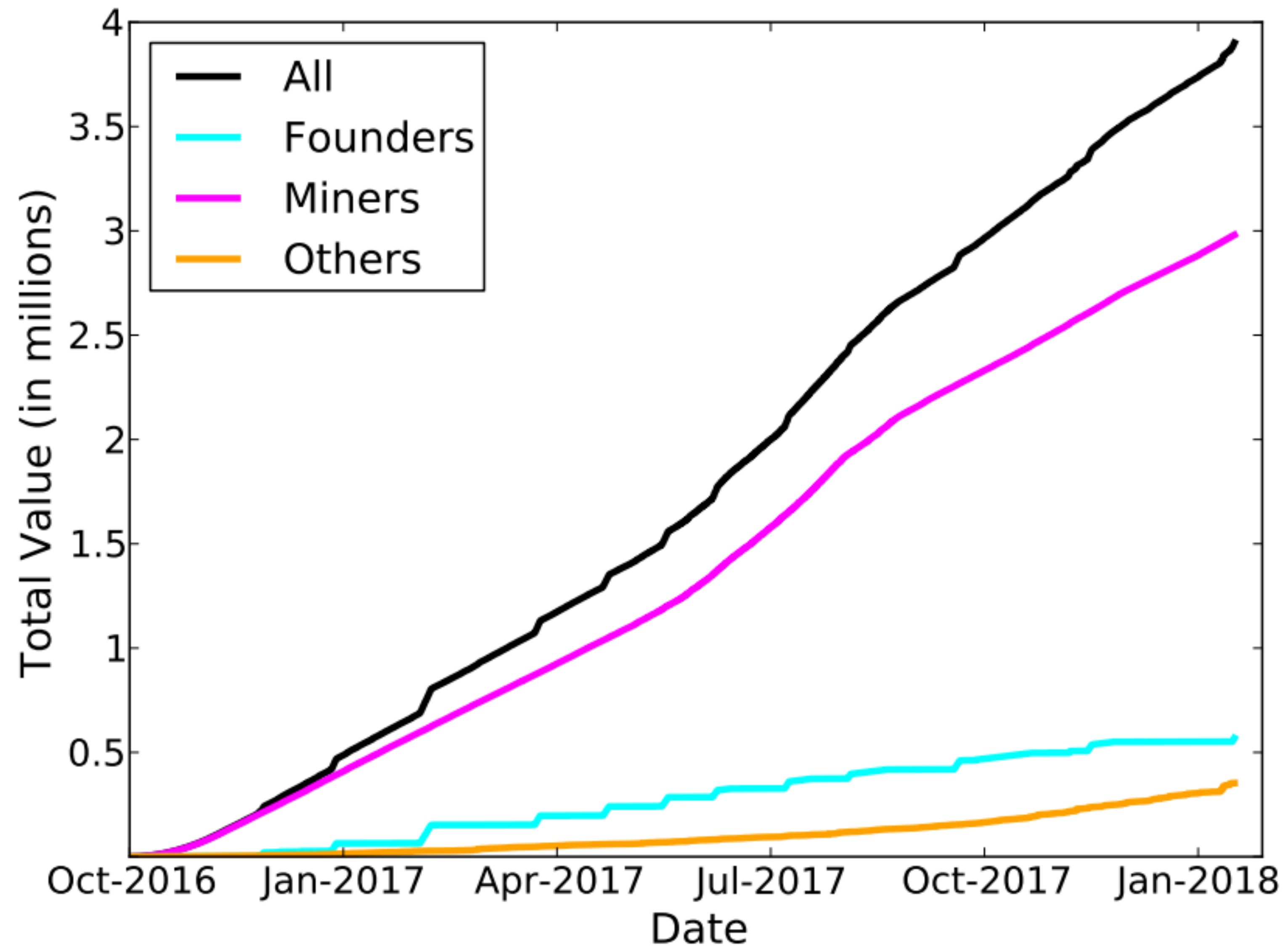


Transparent/public (Value is revealed by both sender and receiver)🔵

Zcash



Zcash



ZK + ML + Blockchain

Private input

The input to the model is hidden, but the model itself is public. This is particularly useful for applications involving sensitive and private data such as financial records, biometric data (e.g., fingerprint, face), medical records, and location information. For example, one can prove he is over 21 years old without disclosing his age. Or an insurance company uses a credit score model for loan approvals. The model is made public for transparency, but the inputs, such as an applicant's salary and bank statements, should be kept confidential.

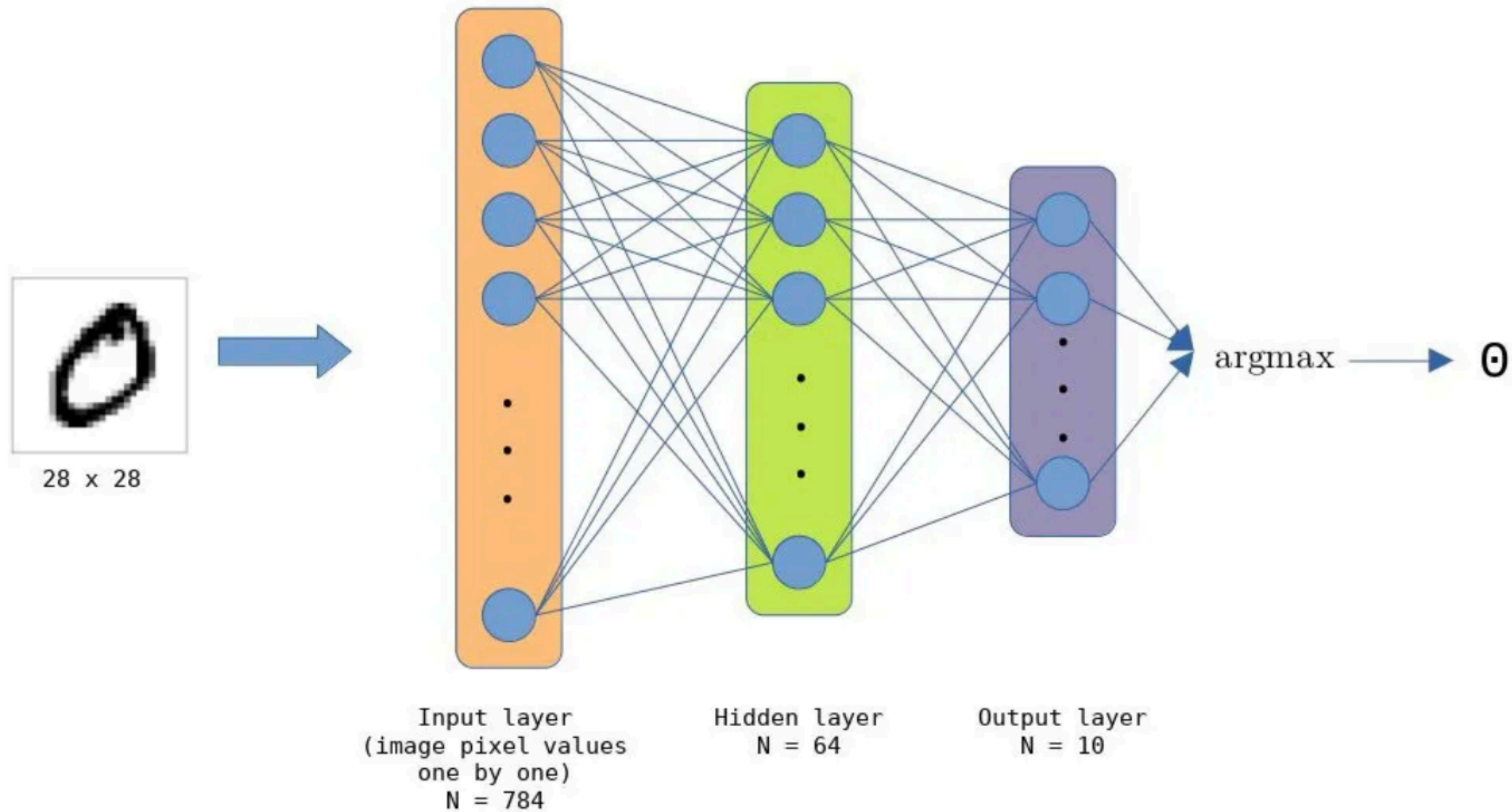
ZK + ML + Blockchain

Private model

The input to the model is public, but the model itself is private, often because it is intellectual property and proprietary. For instance, we use a tumor classification model owned by a private company to detect tumors from images. The model is certified to have 99% accuracy when classifying a public dataset. The company can just publish the cryptographic commitment of its model, i.e., hash of all model parameters. We can be sure the model is legitimate, while not seeing it. The cryptographic commitment also ensures the same model is applied to everyone, for fairness. This is desired in, e.g., an admission model which ranks candidates based on their public information.

ZK + ML + Blockchain

MNIST Dataset



ZK + ML + Blockchain

Private Input

```
def main(private u64[N_NODES_IN] model_inputs
          u64[N_NODES_HL][N_NODES_IN] weights0, \
          u64[N_NODES_OUT][N_NODES_HL] weights1, \
          u64[N_NODES_HL] biases0, \
          u64[N_NODES_OUT] biases1, \) {
    u64[N_NODES_HL] step0 = apply_weights0(model_inputs, weights0);
    u64[N_NODES_HL] step1 = add_biases0(step0, biases0);
    u64[N_NODES_HL] step2 = apply_relu(step1);
    u64[N_NODES_OUT] step3 = apply_weights1(step2, weights1);
    u64[N_NODES_OUT] step4 = add_biases1(step3, biases1);
    u64 res = argmax(step4);

    assert(res == TARGET_CLASS);
    return;
}
```

ZK + ML + Blockchain

Private Model

<https://github.com/sCrypt-Inc/scrypt-ml/blob/master/zk/private-model/zokrates/mnist/circuits/root.zok>

```
def main(private u64[N_NODES_HL][N_NODES_IN] weights0, \
         private u64[N_NODES_OUT][N_NODES_HL] weights1, \
         private u64[N_NODES_HL] biases0, \
         private u64[N_NODES_OUT] biases1, \
         u64[N_TEST_EXAMPLES][N_NODES_IN] test_examples, \
         u64[N_TEST_EXAMPLES] test_labels) {
    u32 mut correct = 0;

    for u32 idx_test_example in 0..N_TEST_EXAMPLES {
        u64[N_NODES_HL] step0 = apply_weights0(test_examples[idx_test_example], weights0);
        u64[N_NODES_HL] step1 = add_biases0(step0, biases0);
        u64[N_NODES_HL] step2 = apply_relu(step1);
        u64[N_NODES_OUT] step3 = apply_weights1(step2, weights1);
        u64[N_NODES_OUT] step4 = add_biases1(step3, biases1);
        u64 res = argmax(step4);

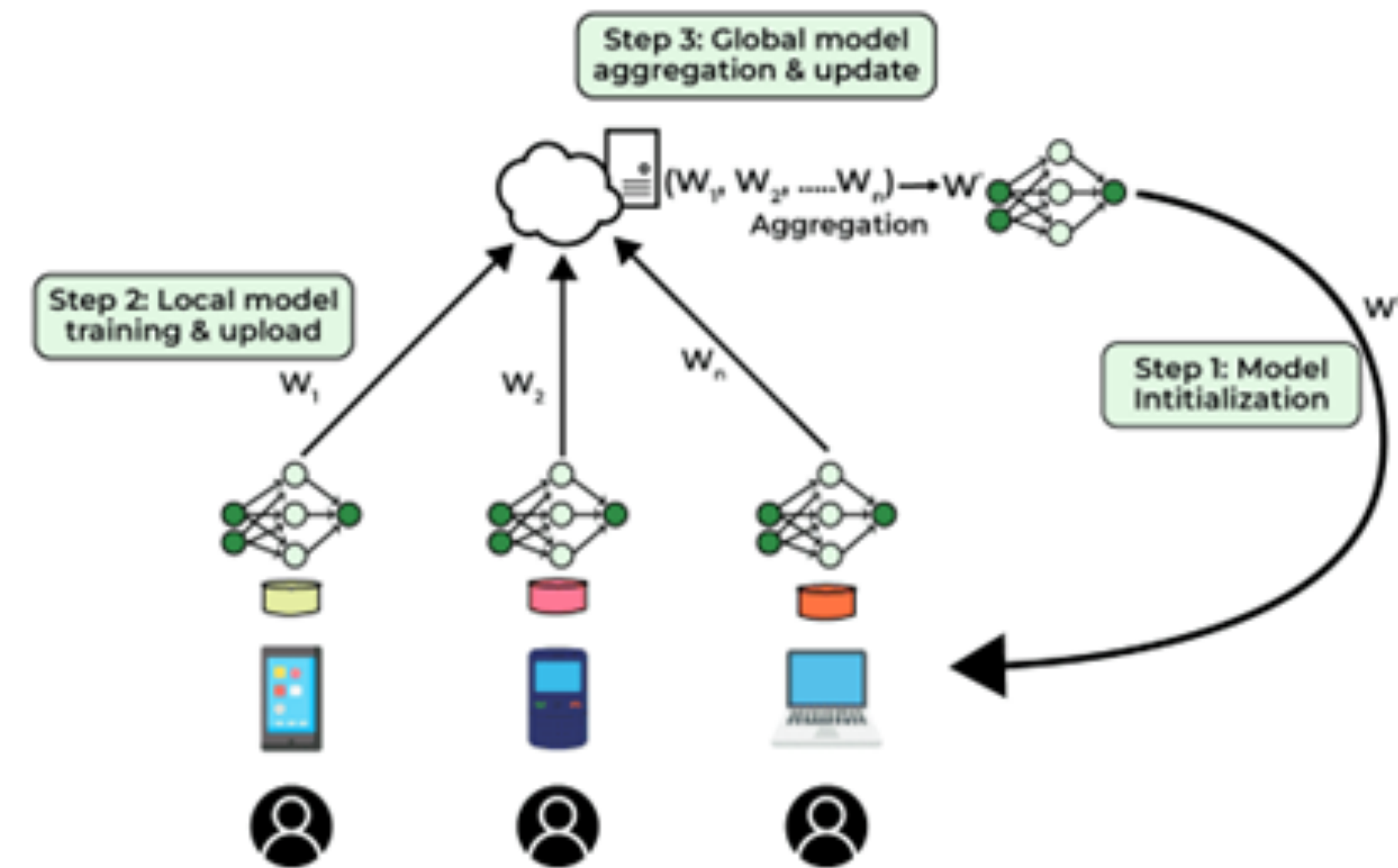
        u32 to_add = if res == test_labels[idx_test_example] { 1 } else { 0 };
        correct = correct + to_add;
    }

    assert((correct * 100) / (N_TEST_EXAMPLES * 100) >= TARGET_CA);
    return;
}
```

ZK + FL + Blockchain

Advancing Blockchain-based Federated Learning through Verifiable Off-chain Computations*

Abstract—Federated learning may be subject to both global aggregation attacks and distributed poisoning attacks. Blockchain technology along with incentive and penalty mechanisms have been suggested to counter these. In this paper, we explore verifiable off-chain computations using zero-knowledge proofs as an alternative to incentive and penalty mechanisms in blockchain-based federated learning. In our solution, learning nodes, in addition to their computational duties, act as off-chain provers submitting proofs to attest computational correctness of parameters that can be verified on the blockchain. We demonstrate and evaluate our solution through a health monitoring use case and proof-of-concept implementation leveraging the ZoKrates language and tools for smart contract-based on-chain model management. Our research introduces verifiability of correctness of learning processes, thus advancing blockchain-based federated learning.



ZK + FL + Blockchain

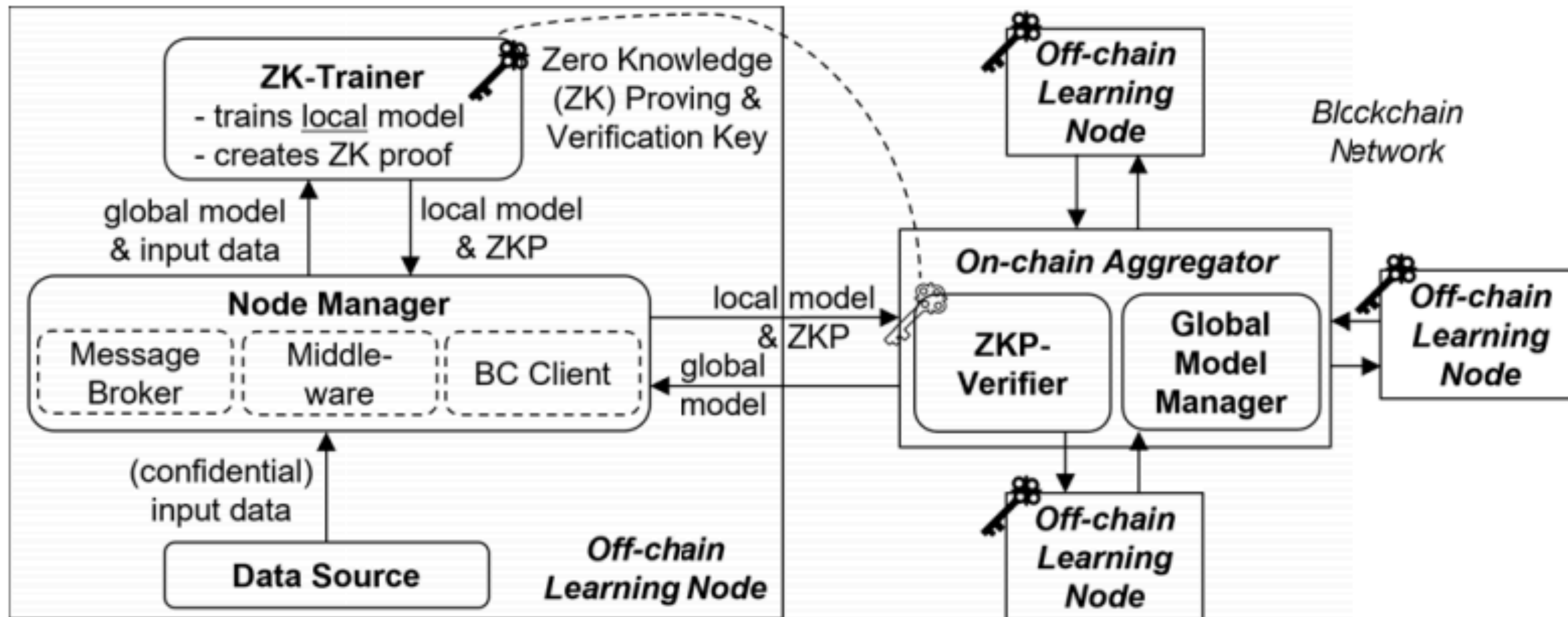
FL: Federated Learning

Federated learning provides an architectural approach to apply machine learning models to a distributed setting where many nodes jointly train a shared global model in an efficient and privacy-preserving manner [19].

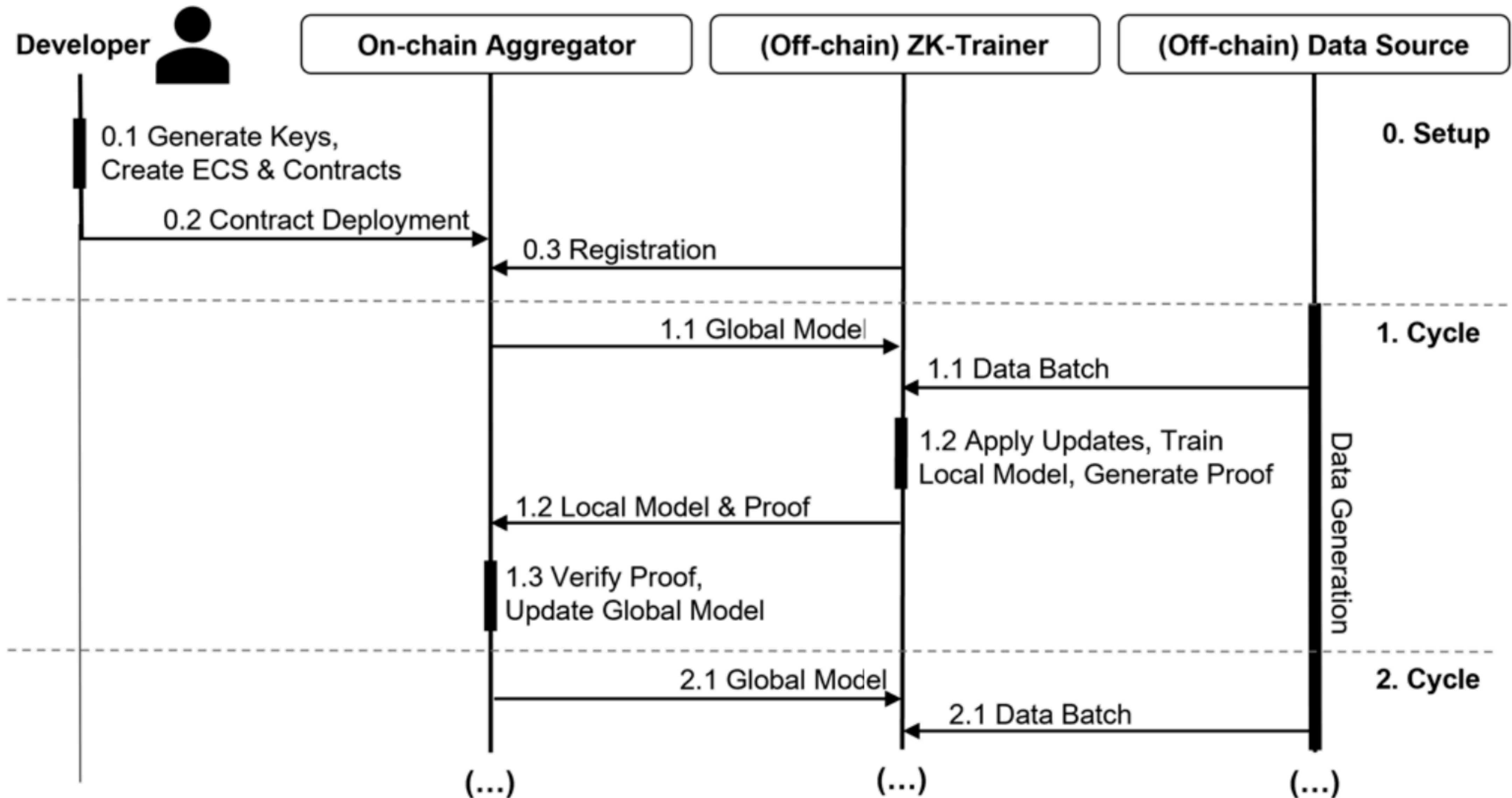
In federated learning, learning parameters are computed as local models by distributed learning nodes on local data. Each participating node provides only its model to an aggregator that combines all received models into a global model which represents the collective learning insights of the learning network. This global model is then locally used by the learning nodes in the next training cycle. Thereby, each learning node benefits from the training executed at other nodes without revealing potentially confidential inputs [19], [21].

ZK + FL + Blockchain

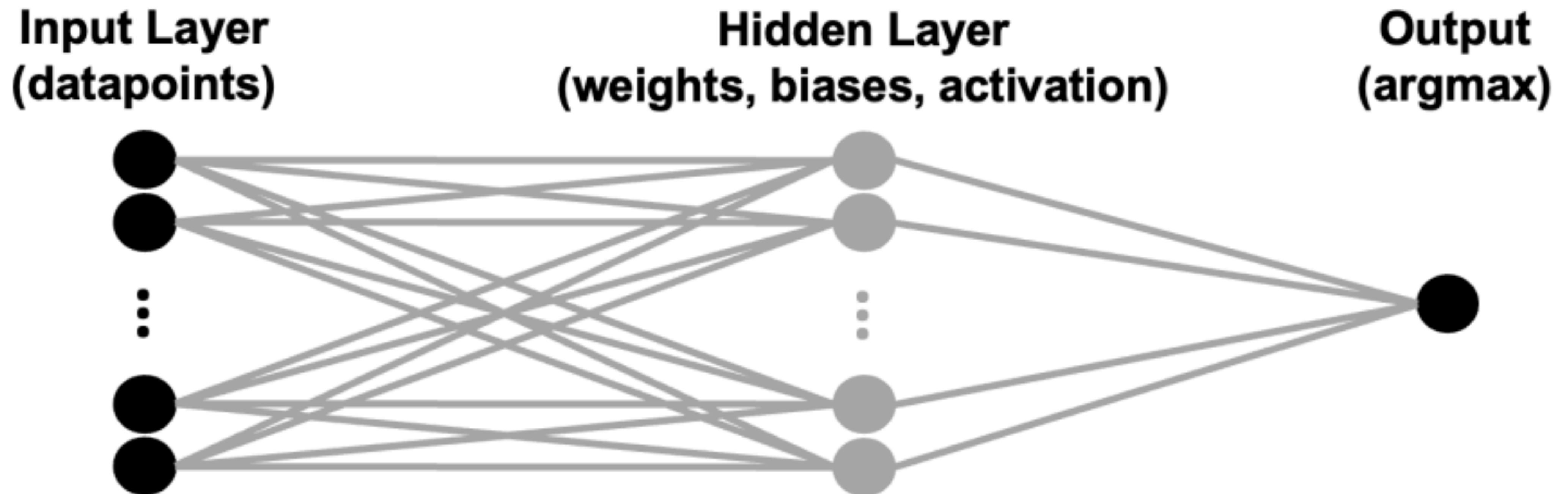
- *Tamper-resistance*: No party can maliciously manipulate a learning model, i.e., neither the local nor the global model.
- *Global verifiability*: The computation of the local and the global model must be verifiable by all participating nodes.



ZK + FL + Blockchain



ZK + FL + Blockchain



ZK + FL + Blockchain

1) Learning Contract: The learning contract provides the updating mechanism for the global model and, as the common endpoint for the off-chain learning nodes, implements the management of the updating cycles introduced in Section III. Analogue to the local model, the global model consists of the weight-vector and the bias-matrix. The updating mechanism requires the smart contract to store two versions: One that is only updated at the end of the cycle and is consumable by off-chain learning nodes. And a temporary one that is used to aggregate the updated local models of the participating learning nodes using a simple moving average function. At the end of an updating cycle, the temporary model replaces the other.

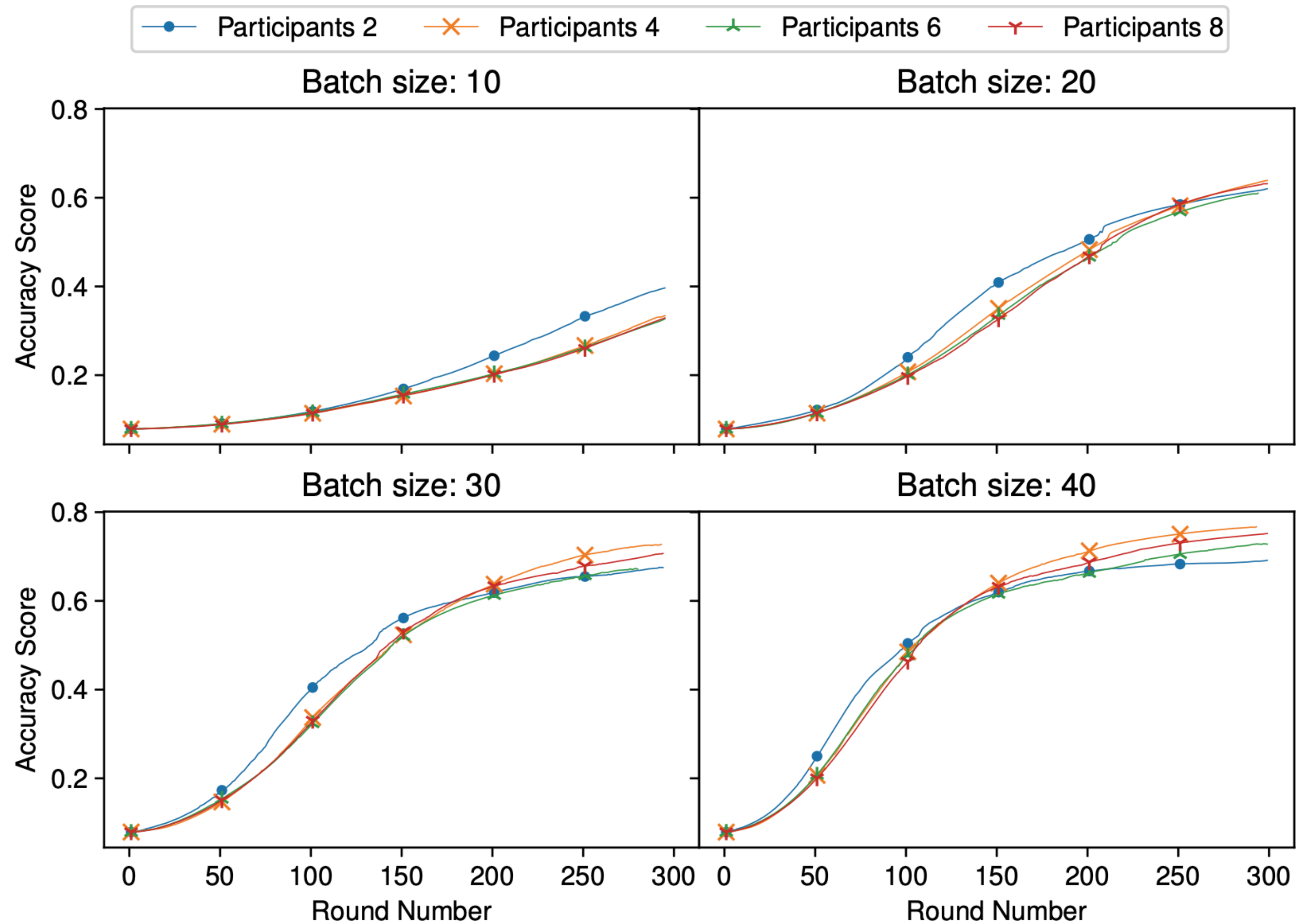
ZK + FL + Blockchain

2) *Verifier Contract*: The verifier contract is called from the learning contract before an update is applied to the global model. It implements the ZKP verification logic in the *verifyTx()* function and treats the verification key as hard-coded parameter. The contract is automatically generated as a Solidity smart contract by means of the ZoKrates CLI.

ZK + FL + Blockchain

Batch size [#]	10	20	30	40
Compile Time [sec]	27.79	56.55	93.01	127.46
<i>STD [sec]</i>	0.02	0.87	3.06	5.70
Compile Memory [GB]	2.26	4.62	6.82	9.31
<i>STD [MB]</i>	80.5	18.9	18.0	56.3
Setup Time [sec]	651.35	1290.40	2107.88	2711.53
<i>STD [sec]</i>	2.00	1.20	7.33	56.87
Setup Memory [GB]	2.02	4.12	6.68	8.28
<i>STD [MB]</i>	23.82	0.00	157.51	114.405
Witness Time [sec]	8.90	19.21	27.43	37.76
<i>STD [sec]</i>	0.50	1.50	0.50	0.50
Witness Memory [GB]	0.95	1.88	2.96	3.72
<i>STD [MB]</i>	2.45	0.00	0.00	0.00
Proof Time [sec]	39.55	78.25	145.50	167.56
<i>STD [sec]</i>	0.03	1.08	1.05	4.34
Proof Memory [GB]	4.39	8.74	13.43	17.45
<i>STD [MB]</i>	0.00	0.00	0.00	0.00

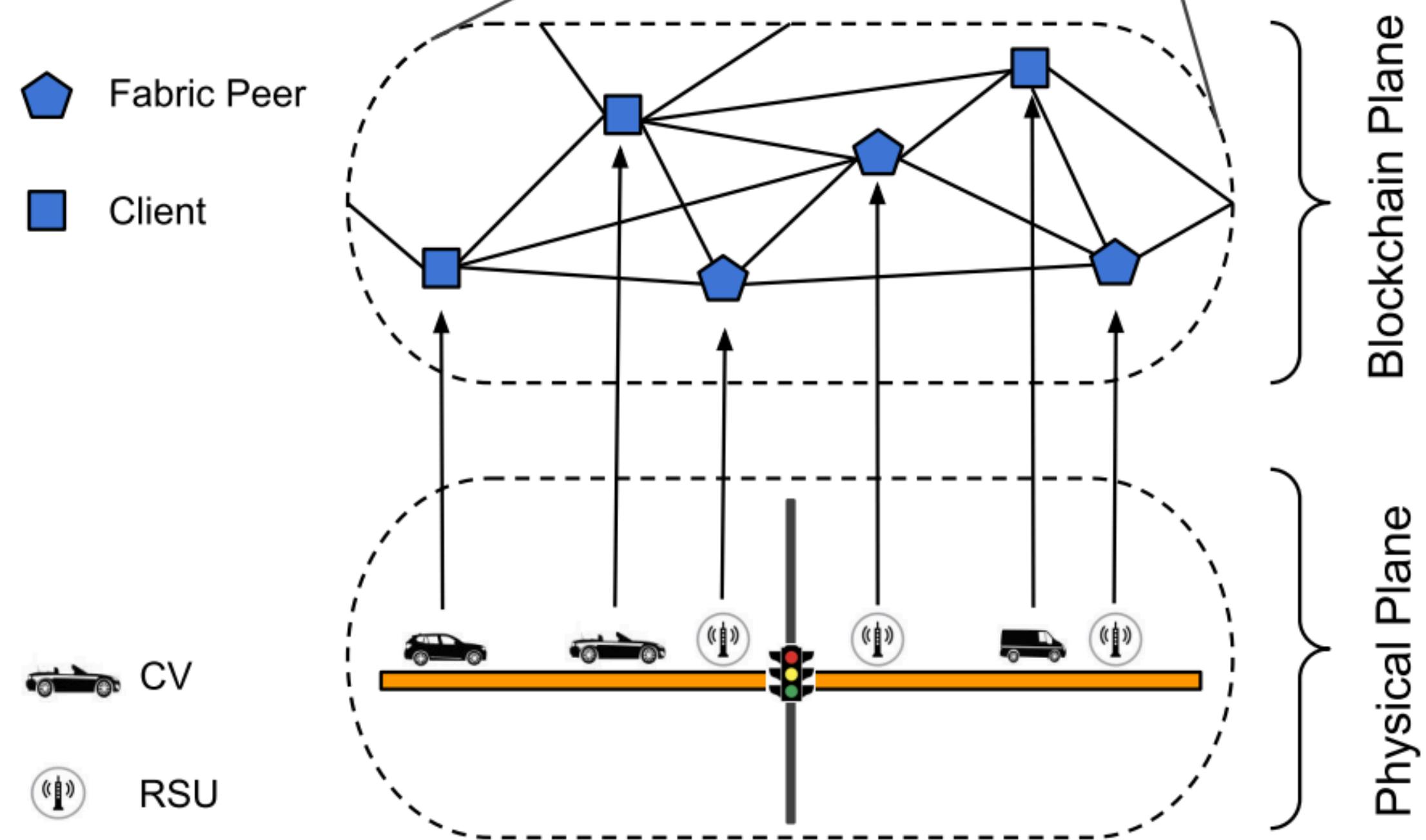
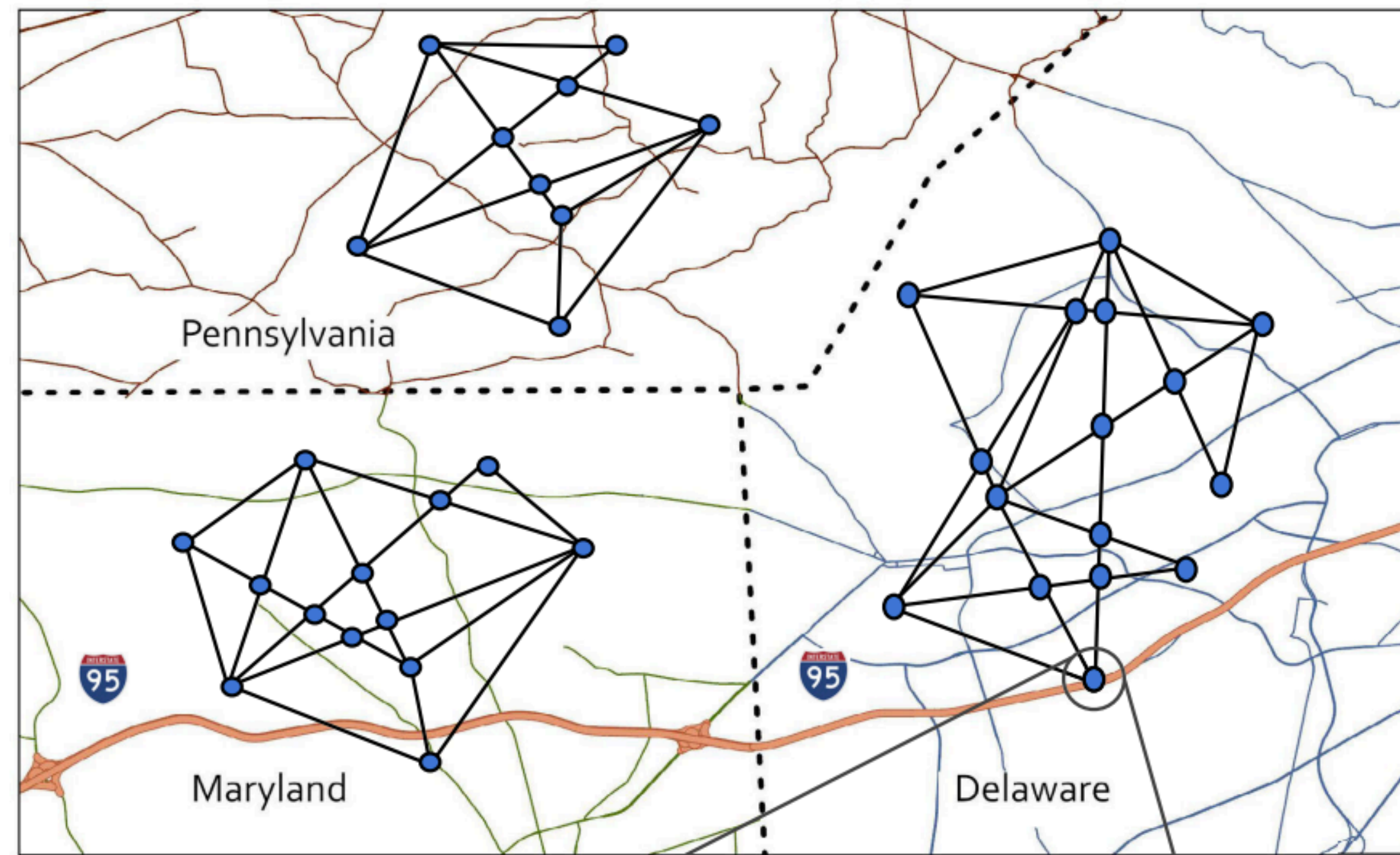
ZK + FL + Blockchain



ZK + Traffic + Blockchain

ABSTRACT Incorporation of connected vehicle (CV) data into real-time traffic management systems presents a host of new challenges resulting from the current lack of data integrity and data privacy in traffic networks. Over the past few years, blockchain technologies have been inspiring extensive innovations in the transportation field. However, due to the transparency property, sensitive data stored on the blockchain would be accessible to anyone, resulting in a lack of privacy. In this paper, we propose a decentralized and location-aware architecture to address the data integrity along with the privacy-preserving issues in blockchain-based traffic management systems. Our proposed architecture integrates with permissioned and modular blockchain network and non-interactive zero-knowledge range proof (ZKRP) protocol. We develop the prototype system on the Hyperledger Fabric platform and Hyperledger Ursa cryptographic library. The performance results show that our approach is effective and feasible for real-time traffic management while preserving the data privacy requirements.

ZK + Traffic + Blockchain



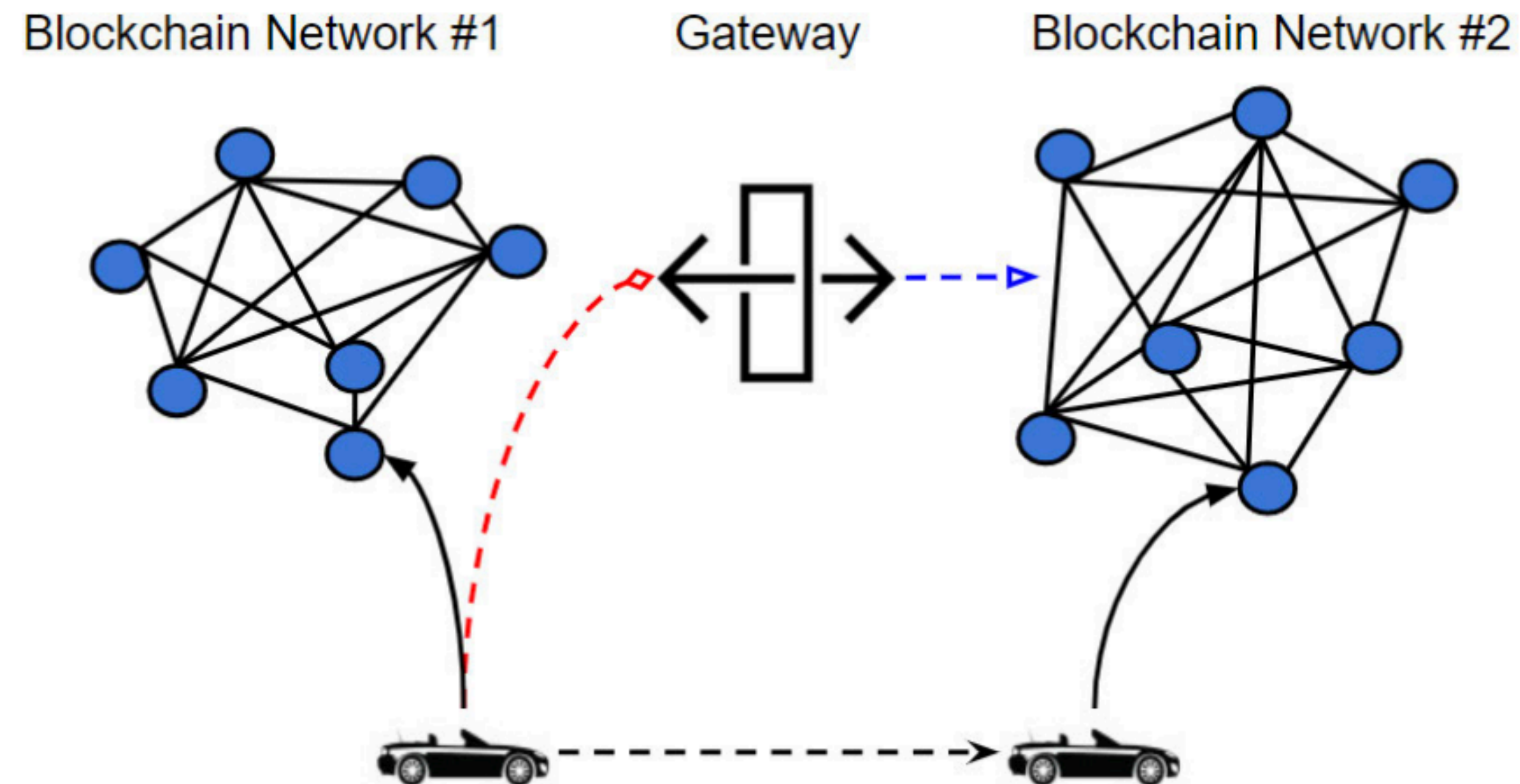
ZK + Traffic + Blockchain

- We present a decentralized and location-aware traffic data management architecture for multiple blockchain-based connected vehicular networks scenario. The architecture provides a novel design of transforming centralized traffic data management systems into decentralized blockchain-based networks and maintained vehicular data as digital records
- We propose the concept of *gateway* that resides between two adjacent blockchain-based traffic management systems to switch traveling vehicles from one blockchain into another. The gateway is responsible for verifying the information of an incoming vehicle, preventing spoofing attacks from malicious vehicles, and logging into the ‘entering’ traffic management system on behalf of the traveling vehicle. Specifically, we articulate the design of the gateway by developing a non-interactive zero-knowledge range proof (ZKRP) scheme, where a traveling vehicle (acting as a prover) sends ZKRP-encrypted message to the gateway (acting as a verifier) to validate the information of the vehicle without revealing any sensitive data.

ZK + Traffic + Blockchain

A. GATEWAY DESIGN WITH ZKRP SCHEME

As shown in Fig. 3, we introduce gateways that are deployed on the boundaries between blockchain-based vehicular networks for seamlessly switching from one blockchain network into another. To preserve the privacy of the traveling vehicle, we describe how to construct the ZKRP protocol step by step for the gateway module. The traversing vehicle acts as a prover to prove its vehicular information (e.g., location) to a gateway, acting as a verifier, in a ZKRP-based encrypted message. As a result, the vehicle passes the boundary and switch the blockchain network without revealing any sensitive information to the other parties.



ZK + Traffic + Blockchain

Scheme Version 1 Interactive ZKP

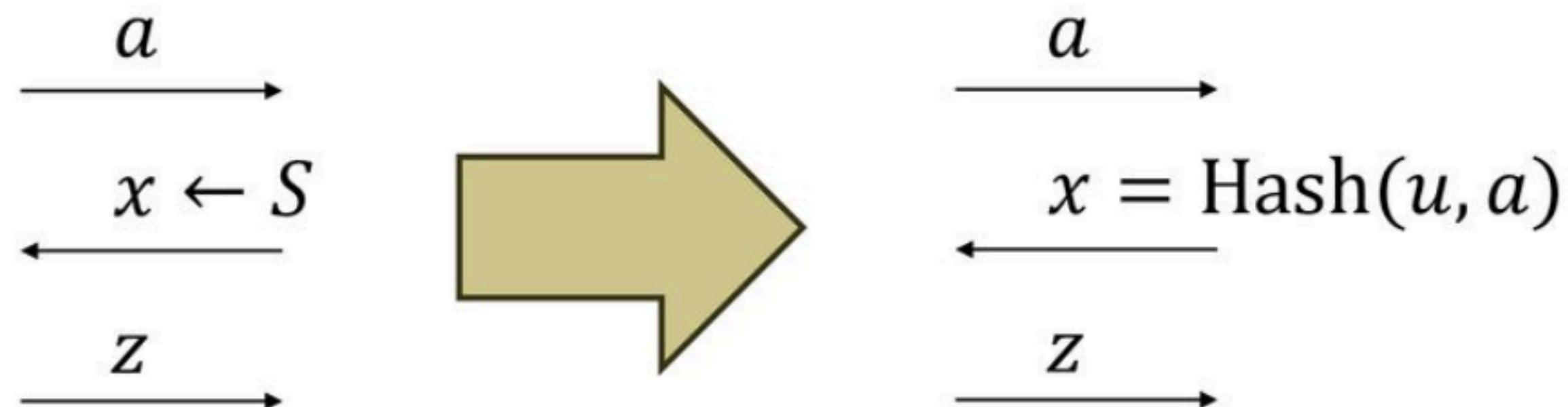
- 1) The traversing CV wants to prove to the gateway that it comes from the location δ : compute the commitment based on the discrete logarithm [24] of $y = g^\delta$ to the base g .
 - 2) The traversing CV picks a random $v \in \mathbb{Z}_p$, computes $t = g^v$ and sends t to the gateway.
 - 3) The gateway picks a random $c \in \mathbb{Z}_p$, and send it back as a challenge message to the traversing CV.
 - 4) The traversing CV computes $r = v - c\delta$ and returns r to the gateway.
 - 5) The gateway checks whether $g^r y^c \equiv t$. This holds because $g^r y^c = g^{v-c\delta} g^{\delta c} = g^v = t$.
-

ZK + Traffic + Blockchain

Fiat-Shamir Heuristic

The security of the Fiat-Shamir conversion relies on hash functions that possess randomness properties to generate unpredictable challenge values. This prevents a dishonest Prover from pre-computing responses or manipulating outcomes. Cryptographic hash functions like SHA2 or SHA3 that offer collision-resistance and one-wayness provide the needed security properties. Their outputs modeled as Verifier's challenges are computationally infeasible for the Prover to anticipate or control.

Statement $u \in L_R$



ZK + Traffic + Blockchain

3.1 The Fiat-Shamir Heuristic

The Fiat-Shamir heuristic is a technique to convert an interactive protocol to a non-interactive proof in the random oracle model. The key idea is that we replace the verifier's random challenge with the value of a hash function (which we model as a random oracle) on the prover's first message and the input. For example, for the Schnorr identification protocol, we obtain the following:

$\mathbf{P}(g, x, h = g^x) :$

$$r \xleftarrow{\mathbb{R}} \mathbb{Z}_q$$

$$u \leftarrow g^r$$

$$c \leftarrow H(g, h, u)$$

$$z \leftarrow r + cx$$

output $\pi = (u, c, z)$

$\mathbf{V}(g, h = g^x, \pi = (u, c, z)) :$

$$c \stackrel{?}{=} H(g, h, u)$$

$$g^z \stackrel{?}{=} u \cdot h^c$$

ZK + Traffic + Blockchain

Scheme Version 2 Non-Interactive ZKP

- 1) The traversing CV wants to prove that it comes from the location δ : compute the commitment based on the discrete logarithm of $y = g^\delta$ to the base g .
 - 2) The traversing CV picks a random $v \in \mathbb{Z}_p$, computes $t = g^v$.
 - 3) The traversing CV computes $c = H(g, y, t)$, where H is a cryptographic hash function [26].
 - 4) The traversing CV computes $r = v - c\delta$. The resulting proof is the pair (t, r) . As r is an exponent of g , it is calculated modulo $q - 1$.
 - 5) The gateway checks whether $g^r y^c \equiv t$.
-

ZK + Traffic + Blockchain

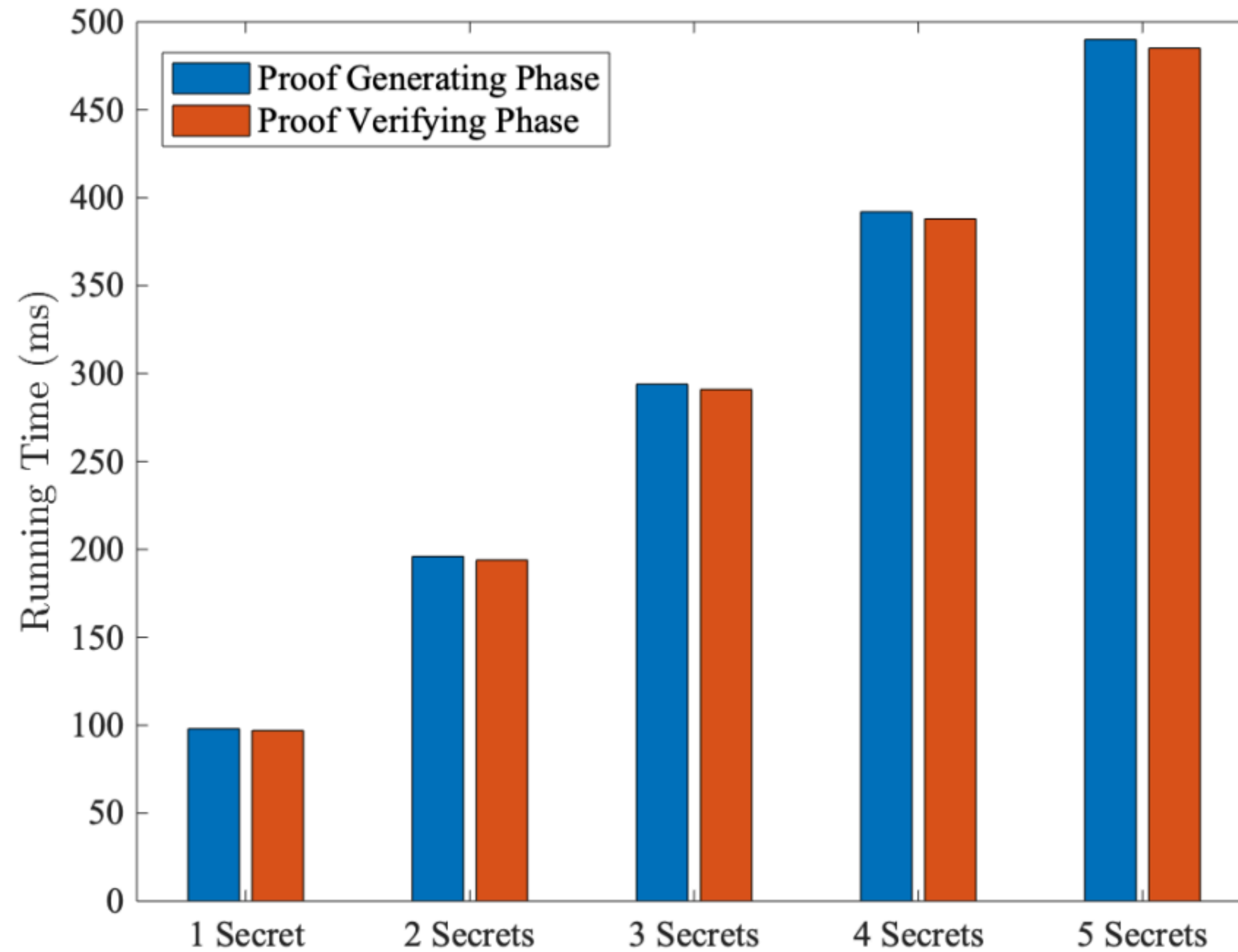


FIGURE 11. Running time of Ursa ZKRP vs. the number of secrets.

ZK + Traffic + Blockchain

